AI News Digest Portal

Technical Documentation

Developer Guide

Version 1.0

Table of Contents

1. Project Overview

Purpose

The AI News Digest Portal is an automated news aggregation and curation platform designed to alleviate information overload in the technology sector. It actively monitors multiple global news APIs, identifies the most relevant articles related to Artificial Intelligence, and utilizes Large Language Models (Google Gemini) to read, summarize, and score the articles for importance. The final output is an easily digestible, highly relevant top-5 daily news summary formatted as an Outlook-compatible `.eml` draft.

Problem Solved

Professionals in fast-moving industries struggle to keep up with the overwhelming volume of daily news articles. Identifying crucial information versus noise takes significant manual effort. This software automates the aggregation, filtering, reading, summarizing, and distribution of industry news, entirely removing the manual curation bottleneck.

Target Users

- **Technology Executives & CTOs**: Seeking high-level, concise summaries of the day's most important AI news.
- **Researchers & Developers**: Tracking industry trends without manually browsing multiple news sites.
- **Content Creators**: Looking for a daily pre-curated list of top stories to publish in newsletters or blogs.

Key Features

- **Multi-Source Aggregation**: Pulls news from 4 independent providers (NewsAPI, MediaStack, NewsData.io, ApiTube) to ensure global coverage.
- **AI-Powered Curation**: Uses Google Gemini to act as a human editor—reading each article, verifying its relevance, writing a concise 3-sentence summary, and assigning an Importance Score (1-10).

- **Automated De-duplication**: Filters out duplicate stories covering the exact same event across different news outlets.
- **Smart Dashboard**: A modern React-based interface that groups curated news by date.
- **One-Click Digest Export**: Instantly exports the day's top 5 articles into a ready-to-send `.eml` email draft.
- **Dynamic Prompts**: Administrators can modify the AI's core instructions via the UI to change the curation focus on the fly.

High-Level Architecture

The system follows a classic decoupled client-server architecture:

- A React Single Page Application (SPA) handles all user interactions and polling.
- A Node.js REST API serves as the central brain. It receives requests from the SPA, delegates heavy background tasks (like contacting external APIs), handles MongoDB transactions, and generates files dynamically.
- External Services act as data feeders (News APIs) and cognitive processors (Google Gemini).

2. System Architecture

The AI News Digest Portal relies on a highly modular, decoupled architecture consisting of four major components:

Frontend

- A React-based Single Page Application (SPA).
- Manages localized state and server state caching.
- Responsible for polling background tasks and enforcing UI-level validation.

Backend

- An Express.js REST API server.
- Acts as the central orchestrator, protecting routes via JWT Middleware.
- Delegates tasks to specific controllers and services.

- Assembles the final `.eml` exports dynamically.

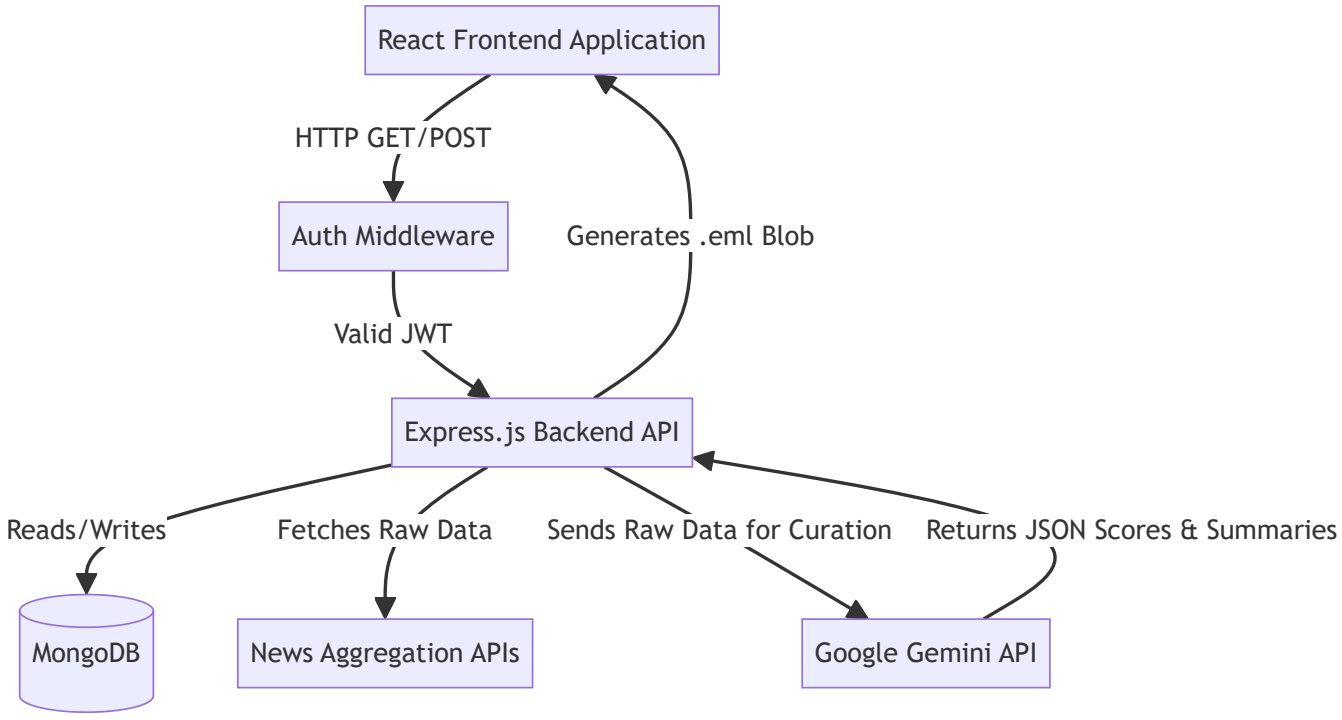
Database

- MongoDB serves as the persistent storage layer.
- Maintains records for users, AI-curated articles, application settings, and historical digest logs.

External Services

- **News Providers**: REST integrations with NewsData.io, ApiTube, NewsAPI, and MediaStack to pull raw data.
- **Cognitive Engine**: REST integration with Google Generative AI (Gemini) to evaluate raw data and provide human-like summaries and scores.

Request Flow and Component Interaction



3. Technology Stack

Frontend Stack

- **React.js & Vite**: Chosen for fast development builds and modern component-based UI construction. Vite provides significantly faster Hot Module Replacement (HMR) than Webpack.
- **TailwindCSS**: Used for rapid UI styling. It provides utility classes that keep styles co-located with components and ensures consistent design without maintaining bloated CSS files.
- **Zustand**: A lightweight, boilerplate-free state management library used for managing global authentication state (current user).
- **React Query**: Handles server state, data fetching, caching, and background synchronization, abstracting away complex data-fetching lifecycle management.
- **Lucide-React**: Provides consistent, clean SVG icons throughout the interface.

Backend Stack

- **Node.js & Express.js**: A robust, non-blocking asynchronous environment perfect for an API that relies heavily on numerous external HTTP requests (I/O heavy).
- **Mongoose**: An Object Data Modeling (ODM) library for MongoDB and Node.js. It manages relationships between data, provides schema validation, and translates between objects in code and the representation of those objects in MongoDB.
- **Bcryptjs**: Used for cryptographically secure password hashing.
- **JSON Web Tokens (JWT)**: A stateless authentication standard that allows the frontend to securely access protected backend routes.

External AI & Services

- **Google Generative AI (Gemini 1.5 Flash)**: Chosen for its speed and massive context window, making it ideal for processing batches of news articles and returning structured JSON.
- **News Providers**: Multiple APIs (NewsData.io, ApiTube, NewsAPI, MediaStack) are used to ensure comprehensive coverage and prevent a single point of failure if one provider rate-limits the application.

4. Project Structure

The repository is structured as a monorepo containing two distinct applications: `backend` and `frontend`.

Repository Organization

```
ai-news-digest-portal/
├── backend/
│   ├── src/
│   │   ├── config/       # Database connection initialization
│   │   ├── controllers/  # Route logic and request orchestration
│   │   ├── middlewares/  # Express middlewares (Auth validation)
│   │   ├── models/       # Mongoose schemas representing DB collections
│   │   ├── routes/       # API route definitions
│   │   ├── services/     # External API communication (Gemini, Email generation)
│   │   └── index.js      # Server entry point
│   ├── .env              # Backend environment variables
│   └── package.json      # Backend dependencies
│
├── frontend/
│   ├── src/
│   │   ├── api/          # Axios instance configuration
│   │   ├── assets/       # Static assets
│   │   ├── layouts/      # Reusable UI shells (e.g., Sidebar navigation)
│   │   ├── pages/        # Main route components (Dashboard, Login, Settings)
│   │   ├── store/        # Zustand global state definition
│   │   ├── App.jsx       # React Router setup
│   │   ├── index.css     # Global styles and Tailwind directives
│   │   └── main.jsx      # React DOM entry point
│   ├── vite.config.js    # Vite bundler configuration
│   └── package.json      # Frontend dependencies
```

Architectural Responsibilities

- **Routes (`backend/src/routes/`)**: Strictly define HTTP paths and instantly map them to controllers. No business logic resides here.

- **Controllers (`backend/src/controllers/`)**: Handle HTTP Request/Response objects, extract parameters, and return JSON or files. They coordinate between models and services.
- **Services (`backend/src/services/`)**: Isolate external communications. For example, Gemini API code resides strictly in `aiService.js`, completely decoupled from Express HTTP contexts.
- **Frontend State**: Local UI state stays in components. Persistent authentication state lives in `useAuthStore.js`. Data fetching state belongs to React Query.

5. Installation & Setup

Prerequisites

- Node.js (v18 or higher recommended)
- MongoDB instance (Local or Atlas)
- Redis server (Local or hosted)
- API Keys for Google Gemini and News Providers

Backend Setup

1. **Install Dependencies**
   Navigate to the backend folder and install the required packages.

   ```
   cd backend
   npm install
   ```

2. **Environment Configuration**
   Create the environment variables file.

   ```
   cp .env.example .env
   ```

   Open the `.env` file and populate it with your database connection strings, Redis configuration, and API keys.

3. **Start the Server**
   Run the backend in development mode (which auto-restarts on file changes).

```
npm run dev
```

The terminal will confirm that the server started and MongoDB is connected.

Frontend Setup

1. **Install Dependencies**
   Open a new terminal window, navigate to the frontend directory, and install the packages.

   ```
   cd frontend
   npm install
   ```

2. **Start the Application**
   Run the Vite development server.

   ```
   npm run dev
   ```

   Navigate to `http://localhost:5173` in your web browser.

Initial Application Boot

1. Access the frontend application.
2. Click **Register** to create the initial admin account.
3. Upon registration, you are redirected to the Dashboard.
4. Navigate to **Settings** to configure the AI Prompt and default receiver emails.

6. Configuration

Environment Variables

The application's behavior is dictated by environment variables. The backend requires a `.env` file for all sensitive connections.

- **Server**: `PORT` (Defaults to 5000)

- **Database**: `MONGO_URI` (MongoDB connection string)
- **Redis**: `REDIS_HOST`, `REDIS_PORT`, `REDIS_USERNAME`, `REDIS_PASSWORD`
- **Security**: `JWT_SECRET` (Cryptographic key for signing tokens)
- **AI Integration**: `GEMINI_API_KEY`
- **News APIs**: `NEWSDATA_API_KEY`, `APITUBE_API_KEY`, `NEWSAPI_API_KEY`, `MEDIASTACK_API_KEY`

Note: The frontend operates without environment variables locally, defaulting API calls to `http://localhost:5000/api`.

Database Configuration (Settings Model)

Application-specific behaviors are configured via the database using a strict singleton `Settings` document.

- **Default AI Prompt**: The exact instruction set sent to Gemini. This can be modified via the frontend UI without restarting the server.
- **Receiver Emails**: An array of target email addresses automatically injected into the generated `.eml` drafts.

7. Backend Documentation

The backend is organized by domain modules following a Controller-Service-Model pattern.

Authentication Module

- **Controller (`authController.js`)**: Manages user registration and login workflows. It hashes passwords, verifies credentials, and issues JSON Web Tokens.
- **Middleware (`authMiddleware.js`)**: Intercepts protected routes, extracts the Bearer token from the `Authorization` header, verifies the cryptographic signature, and appends the decoded user object to the request.

News Collection Module

- **Controller (`newsController.js`)**: The core orchestrator for news gathering. It coordinates fetching raw data from multiple providers, de-duplicates articles based on URLs, and batches

them.

- **Service (`aiService.js`)**: Abstracts the Google Gemini integration. It constructs prompts, sends article batches to the AI, and safely parses the returned JSON containing summaries and importance scores.

Digest Module

- **Controller (`digestController.js`)**: Responsible for compiling selected articles into an email format.
- **Service (`emailService.js`)**: Generates the raw RFC 822 (`.eml`) blob. It formats the HTML body, injects the receiver emails, and sets the `X-Unsent: 1` header to ensure the file opens as a draft in client mail applications.

Settings Module

- **Controller (`settingsController.js`)**: Manages the singleton settings document, ensuring administrators can update prompts and configurations at runtime.

8. Frontend Documentation

Application Structure

The frontend is a React application built with Vite. It relies on a component-based architecture where pages represent full views and layouts provide structural consistency.

Routing & Layouts

Routing is managed by React Router.

- **Public Routes**: `/login` , `/register` .
- **Protected Routes**: Wrapped in an authentication guard component. If a user is not authenticated, they are redirected to `/login` .
- **MainLayout**: A structural component providing a persistent sidebar navigation shell across all protected views (Dashboard, Settings, History).

State Management

- **Zustand**: Manages global ephemeral state. Specifically, the `useAuthStore.js` tracks the currently logged-in user and their JWT, persisting it to `localStorage` to survive page refreshes.
- **React Query**: Handles server state. When the Dashboard requests news articles, React Query manages the loading state, caches the response, and provides mechanisms for background refetching.

API Communication

All backend communication occurs via a centralized Axios instance (`api/axios.js`). This instance features an interceptor that automatically attaches the user's JWT as a Bearer token to every outgoing request. It also centralizes error handling for 401 Unauthorized responses.

9. Database Design

The application utilizes four primary Mongoose schemas.

1. User (`models/User.js`)

Stores authentication credentials.

- **Fields**: `name` , `email` (Unique), `password` , `isAdmin` .
- **Hooks**: Uses a `pre('save')` hook to automatically hash passwords using `bcryptjs` before database insertion.

2. Article (`models/Article.js`)

Stores AI-curated news articles.

- **Fields**: `title` , `url` (Unique index prevents duplicate scraping), `source` , `publishedAt` , `summary` , `importanceScore` , `status` (Pending, Sent, Rejected).
- **Indexes**: Compound index on `{ status: 1, publishedAt: -1 }` to optimize Dashboard grouping and retrieval queries.

3. Digest (`models/Digest.js`)

Serves as an audit log for generated `.eml` exports.

- **Fields**: `sentBy` (Reference to User), `articles` (Array of References to Article), `status` .

4. Settings (`models/Settings.js`)

A singleton document for global application state.

- **Fields**: `defaultPrompt` , `receiverEmails` .
- **Lifecycle**: Seeded automatically at server startup if no document exists, ensuring the application always has a baseline configuration.

10. API Documentation

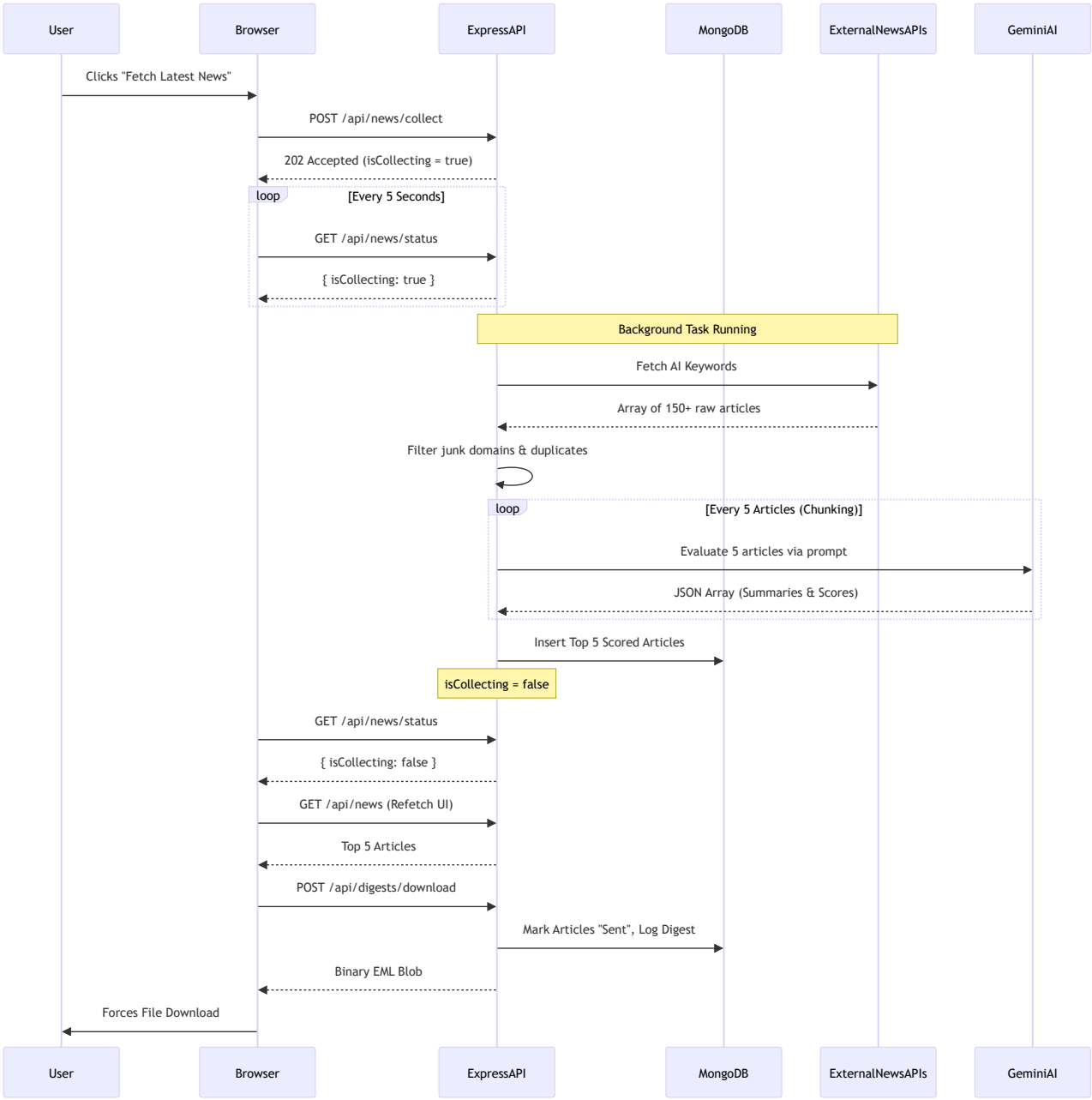
All routes are prefixed with `/api`.

Method	Endpoint	Purpose	Auth Required	Response
POST	`/auth/register`	Creates a new user account.	No	`201 { token, user }`
POST	`/auth/login`	Authenticates a user.	No	`200 { token, user }`
POST	`/news/collect`	Triggers background news gathering.	Yes (Admin)	`202 { message: "Started" }`
GET	`/news/status`	Polls if the background task is running.	Yes	`200 { isCollecting: boolean }`
GET	`/news`	Retrieves curated news articles.	Yes	`200 { articles, page, total }`
POST	`/digests/download`	Generates `.eml` draft for selected IDs.	Yes (Admin)	`200` Binary Blob attachment
GET	`/digests`	Retrieves historical digest generation logs.	Yes (Admin)	`200 { digests }`
GET	`/settings`	Fetches global application settings.	Yes	`200 { settings }`
PUT	`/settings`	Updates global application settings.	Yes (Admin)	`200 { settings }`

11. Core Workflows

Application Polling & Curation Lifecycle

This workflow demonstrates how the system handles long-running AI tasks asynchronously to prevent HTTP timeouts.



12. Error Handling

Backend Recovery Mechanisms

- **General Controllers**: Every controller is wrapped in standard `try/catch` blocks. Uncaught asynchronous errors trigger a `500 Internal Server Error` containing the error message.
- **AI Rate Limiting (Gemini)**: Google Gemini frequently throws `503` (Service Unavailable) or `429` (Too Many Requests) when overwhelmed. The `aiService.js` loop catches these errors at the chunk level. Instead of crashing the entire collection process, it logs the failure for that specific 5-article batch and gracefully continues to the next batch. This guarantees partial success even during API turbulence.
- **AI JSON Parsing**: If the AI returns JSON improperly wrapped in markdown blocks (e.g., ```` ```json ````), a Regex recovery block cleans the string before parsing it.

Frontend Error Handling

- **Axios Interceptors**: The global Axios instance intercepts responses. If a `401 Unauthorized` is detected (e.g., token expired), it can trigger a global logout.
- **UI Feedback**: React Query's built-in `isError` flags are utilized to render visual error alerts in the UI using Tailwind styling.

Database Validations

- Mongoose strictly enforces data boundaries. For instance, attempting to write an invalid `status` string to the `Digest` model results in a Mongoose `ValidationError`, preventing corrupted data states.

13. Security

Authentication and Authorization

- **Password Hashing**: Passwords are never stored in plain text. A Mongoose `pre('save')` hook intercepts account creations or modifications and applies a secure `bcryptjs` hash with a

randomized salt.

- **Token Verification**: JSON Web Tokens are used for sessionless authentication. Tokens are cryptographically signed using the backend's `JWT_SECRET`. Tampered tokens will instantly fail cryptographic validation in the `authMiddleware.js`.

Network Security

- **Cross-Origin Resource Sharing (CORS)**: The backend enables CORS, allowing the React frontend to communicate with it. In production, this must be restricted to the specific origin of the deployed frontend domain.
- **API Key Concealment**: No external API keys (Gemini, NewsAPI) are ever exposed to the client browser. The backend securely manages all external communications server-side.

Safe Downloads

- Browsers may issue a generic safety warning when downloading `.eml` files generated dynamically via `blob:` URLs. Because these files are generated internally by a trusted Node server based on database records, they pose no drive-by-download threat.

14. Deployment

Frontend Deployment

The React frontend must be compiled into static HTML/CSS/JS before deployment.

```
cd frontend
npm run build
```

The resulting `dist/` folder contains purely static files. These can be hosted on Content Delivery Networks (CDNs) or static hosts like Vercel, Netlify, or an AWS S3 Bucket fronted by CloudFront. *Ensure the API base URL points to the production backend domain before building.*

Backend Deployment

The Node.js backend requires a persistent server environment (e.g., an AWS EC2 instance or a DigitalOcean Droplet).

1. **Process Management**: Use PM2 to run the Node application. It ensures the application runs in the background and restarts automatically on failure or server reboot.

   ```
   pm2 start src/index.js --name "ai-news-api"
   pm2 save
   pm2 startup
   ```

2. **Reverse Proxy**: Configure Nginx to reverse proxy port 80/443 traffic to the local Node port (e.g., `5000`).
3. **SSL**: Secure the API using Let's Encrypt (`certbot`) for HTTPS encryption.

Database Hosting

For production environments, utilize a managed Database-as-a-Service like **MongoDB Atlas**. Secure the database by whitelisting the static IP address of your backend server in the Atlas network settings.

15. Extending the Project

Adding New News APIs

To integrate a new news provider:

1. Obtain the API Key and add it to the backend `.env` file.
2. In `backend/src/controllers/newsController.js`, add a new asynchronous fetch function targeting the new provider's endpoint.
3. Map the provider's unique JSON response format to the standardized object expected by the application (`{ title, url, source, publishedAt }`).
4. Append the results to the unified array before passing them to the AI service.

Modifying AI Behavior

The AI's criteria for selecting and summarizing news is dictated entirely by the prompt. Administrators can alter this behavior (e.g., focus on "Cybersecurity" instead of "Generative AI") via the frontend

Settings page. No code deployment is required to change the AI's core logic.

Modifying Database Schemas

To track additional information (e.g., "Author" for an article):

1. Update `backend/src/models/Article.js` to include the new field.
2. Ensure the news scraping functions in `newsController.js` capture this field.
3. Update the frontend UI components in the `Dashboard` to render the new data.

16. Troubleshooting

- **1. MongoDB Connection Refused (`MongoNetworkError`)**
  - **Cause**: The `MONGO_URI` is missing, invalid, or the server's IP is not whitelisted in MongoDB Atlas.
  - **Solution**: Verify the `.env` file. In Atlas, ensure "Network Access" includes your current IP address.
- **2. "Error processing batch: 503" or "429 Too Many Requests"**
  - **Cause**: Google's Gemini API is actively rate-limiting your requests.
  - **Solution**: Wait before initiating another news collection. If persistent in production, consider upgrading the Google AI Studio account tier for a higher quota.
- **3. Empty Dashboard After Successful Collection**
  - **Cause**: The AI evaluated the articles but rejected all of them because they did not meet the relevance criteria defined in the prompt.
  - **Solution**: Navigate to Settings and ensure the AI Prompt is not overly restrictive.
- **4. Browser Blocks .EML Download**
  - **Cause**: Browsers inherently distrust `.eml` files generated via `blob:` URLs without direct anchor tag navigation.
  - **Solution**: Users can safely click "Keep" on the browser warning, or utilize the "Redownload Digest" feature in the application's history.
- **5. Login Fails (401 Unauthorized)**
  - **Cause**: The `JWT_SECRET` was modified, invalidating existing tokens, or the user's database record was deleted.

- **Solution**: Re-register the account or revert the `JWT_SECRET` change.

17. Appendix

Full Directory Tree

```
ai-news-digest-portal
├── backend
│   ├── .env
│   ├── .gitignore
│   ├── Dockerfile
│   ├── package.json
│   ├── src
│   │   ├── config
│   │   ├── controllers
│   │   ├── middlewares
│   │   ├── models
│   │   ├── routes
│   │   ├── services
│   │   └── index.js
├── docker-compose.yml
├── docs
│   ├── (Markdown documentation files)
├── frontend
│   ├── .env
│   ├── .gitignore
│   ├── Dockerfile
│   ├── index.html
│   ├── package.json
│   ├── postcss.config.js
│   ├── public
│   ├── src
│   │   ├── api
│   │   ├── assets
│   │   ├── layouts
│   │   ├── pages
│   │   ├── store
│   │   ├── App.jsx
│   │   ├── index.css
│   │   └── main.jsx
```

```
│   ├── tailwind.config.js
│   └── vite.config.js
└── README.md
```

Dependency Overview

- **Backend**: `express`, `mongoose`, `bcryptjs`, `jsonwebtoken`, `@google/genai`, `cors`, `dotenv`.
- **Frontend**: `react`, `react-dom`, `react-router-dom`, `zustand`, `@tanstack/react-query`, `axios`, `lucide-react`, `tailwindcss`, `vite`.